3.Write the python program for Water Jug Problem

Aim:

To write a python program for water jug problem.

Algorithm:

Step1: Start with two empty jugs of different capacities, and a target quantity of water to measure.

Step2: Fill one jug to its maximum capacity.

Step3: Pour the water from the full jug into the other jug until it is full or the first jug is empty.

Step4: If the second jug is full, pour out the water to empty it.

Step5: Repeat steps 2-4, filling and pouring water between the two jugs, until the desired quantity of water is measured or all possible combinations are tried.

Program:

```
from collections import deque
def water_jug_problem(x, y, z):
  """
  Solves the water jug problem using breadth-first search.
  x: size of jug 1     y:
size of jug 2     z: target
amount of water
  Returns the sequence of steps to reach the target amount of water.
  """
  queue = deque([(0, 0, [])])  # Start with both jugs empty
visited = set()     while queue:
    a, b, steps = queue.popleft()
```

```
    if (a, b) in visited:
continue
((a, b))visited.add((a, b))
if a == z or b == z:
      return steps
   queue.append((x, b, steps + ['fill jug 1']))
queue.append((a, y, steps + ['fill jug 2']))
queue.append((0, b, steps + ['empty jug 1']))
queue.append((a, 0, steps + ['empty jug 2']))
amount = min(a, y - b)
   queue.append((a - amount, b + amount, steps + ['pour jug 1 into jug 2']))     #
Pour jug 2 into jug 1
amount = min(x - a, b)
   queue.append((a + amount, b - amount, steps + ['pour jug 2 into jug 1']))
return None


steps = water_jug_problem(4, 3, 2)

if steps:

  print('\n'.join(steps))

else:

  print('No solution found.')
```

OUTPUT:

```
===== RESTART: C:\Users\MY PC\OneDrive\Documents\CSA1709-AI\3. Water Jug.py ====
Fill jug 2
Pour jug 2 into jug 1
Fill jug 2
Pour jug 2 into jug 1
>
```

Result: The Water Jug Problem was successfully resolved using the Breadth-First Search (BFS) state-space algorithm

4. Write the python program for Cript-Arithmetic problem

Aim:

To write a python program for Cript-Arithematic problem.

Algorithm:

Step 1: Write down the arithmetic problem in full, including any carry-overs.

Step 2: Replace each letter with a unique digit from 0 to 9.

Step 3: Generate all possible combinations of digits for each letter, subject to any constraints

'from the carry-overs and the rules of arithmetic.

Step 4: For each combination of digits, evaluate the arithmetic problem and check if it satisfies the given constraints.

Step 5: If a solution is found, output the values of the letters and the arithmetic problem, and

exit the algorithm.

Stepm6: If no solution is found, output a message indicating that no solution exists.

Program:

```
def solve_cryptarithmetic(puzzle):
  """
 Solves the cryptarithmetic puzzle.
 "WORD + WORD = RESULT"    puzzle: a string containing the puzzle in the form of "WORD + WORD = RESULT" Returns a dictionary mapping letters to digits, or None if no solution is found.
  """
 words = puzzle.split()
letters = set(''.join(words))
if len(letters) > 10:
    return None  # More than 10 letters, no solution is possible
for perm in permutations(range(10), len(letters)):
    mapping = dict(zip(letters, perm))
    if all(mapping[word[0]] != 0 for word in words):  # Check for leading zeros
```

```
a= sum(mapping[c] * (10 ** (len(word) - i - 1)) for word in words for i, c in
 enumerate(word[::-1]))
b a %= 10 = a // 10            a %= 10
c= sum(mapping[c] * (10 ** (len(words[2]) - i - 1)) for i, c in enumerate(words[2][::-1]))
      if a == mapping[words[2][-1]] and b + c == sum(mapping[c] * (10 ** (len(word) - i - 1))
for word in words[:-1]):                return mapping     return None # Example usage puzzle =
"SEND + MORE = MONEY"
mapping = solve_cryptarithmetic(puzzle)
if mapping:
  print(mapping)
  print(puzzle.translate(str.maketrans(mapping)))
else:
   print('No solution found.')
```

OUTPUT:

```
== RESTART: C:\Users\MY PC\OneDrive\Documents\CSA1709-AI\4.Cript-Arithmetic.py =
Solution mapping: {'R': 8, 'M': 1, 'Y': 2, 'O': 0, 'S': 9, 'D': 7, 'E': 5, 'N':
6}
Verification: 10652 = 10652
>
```

Result:

The Crypt-Arithmetic Problem was successfully resolved using the Backtracking Search algorithm

5. Write the python program for Missionaries Cannibal problem

Aim:

To write a python program for Missionaries Cannibal problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search. Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Program:

```
from typing import List, Tuple from collections import deque
def is_valid_state(state: Tuple[int, int, int, int, int, int]) ->
bool:
    """
```

```
    Check if a state is valid according to the problem constraints.
    """
    m1, c1, b, m2, c2, _ = state
    if m1 < 0 or c1 < 0 or m2 < 0 or c2 < 0:
        return False     if (m1 > 0 and m1 < c1) or (m2
> 0 and m2 < c2):
        return False
    return True
def get_successors(state: Tuple[int, int, int, int, int, int]) -> List[Tuple[int, int, int, int, int,
int]]:
    """
    Generate all possible valid states that can be reached from a given state.
    """
    m1, c1, b, m2, c2, d =
state     successors = []     if
d == 1:
        # boat is on the left side
    for i in range(3):             for j in
range(3):                 if i+j >= 1
and i+j <= 2:
                new_state = (m1-i, c1-j, 0, m2+i, c2+j, 1)                    if
is_valid_state(new_state):
                successors.append(new_state)
    else:
        # boat is on the right side
    for i in range(3):             for j in
range(3):                 if i+j >= 1
and i+j <= 2:
```

```
            new_state = (m1+i, c1+j, 1, m2-i, c2-j, 0)                    if
is_valid_state(new_state):
            successors.append(new_state)
  return successors

def breadth_first_search() -> List[Tuple[int, int, int, int, int, int]]:
"""
  Find the solution using a breadth-first search algorithm.
   """
   initial_state = (3, 3, 1, 0, 0, 1)
goal_state = (0, 0, 0, 3, 3, 0)
visited = set()
  queue = deque([(initial_state, [])])
while queue:
    state, path = queue.popleft()        if
state == goal_state:             return path +
[state]        visited.add(state)        for
successor in get_successors(state):
if successor not in visited:
        queue.append((successor,
path+[state]))     return [] if __name__ ==
'__main__':
  solution =
breadth_first_search()
print("Solution:")     for state in
solution:
    print(state)
```

Output:

```
= RESTART: C:\Users\MY PC\OneDrive\Documents\CSA1709-AI\5. Missionaries Cannibal
.py
Solution:
(3, 3, 0, 0, 1)
(3, 1, 0, 2, 0)
(3, 2, 0, 1, 1)
(3, 0, 0, 3, 0)
(3, 1, 0, 2, 1)
(1, 1, 2, 2, 0)
(2, 2, 1, 1, 1)
(0, 2, 3, 1, 0)
(0, 3, 3, 0, 1)
(0, 1, 3, 2, 0)
(0, 2, 3, 1, 1)
(0, 0, 3, 3, 0)
>
```

Result:

The Missionaries Cannibal problem was successfully resolved using the Breadth-First Search (BFS) algorithm

6. Write the python program for Vacuum Cleaner problem

Aim:

To write a python program for vacuum cleaner problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or dirty).
Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.
Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.
Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).
Step 5: Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).
Step 6: If the successor state is valid, add it to the search tree and continue the search. Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.
Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program:

```
from typing import List, Tuple

def get_successors(state: Tuple[Tuple[int, int], List[List[int]]]) -> List[Tuple[Tuple[int, int], List[List[int]]]]:
    """
    Generate all possible valid states that can be reached from a given state.
    """
    position, grid = state
    successors = []

    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        x, y = position
        new_x, new_y = x + dx, y + dy
        if 0 <= new_x < len(grid) and 0 <= new_y < len(grid[0]):
```

```
        new_grid = [row[:] for row in grid]
if new_grid[new_x][new_y] == 1:
new_grid[new_x][new_y] = 0
            successors.append(((new_x, new_y), new_grid))     return successors def
depth_first_search(state: Tuple[Tuple[int, int], List[List[int]]], visited: set) -> bool:
"""
  Find the solution using a depth-first search algorithm.
   """     if all(all(cell == 0 for cell in row) for row in
state[1]):
    return True     visited.add(state)     for
successor in get_successors(state):         if
successor not in visited:             if
depth_first_search(successor, visited):
          return True
return False if __name__
== '__main__':     grid =
[[0, 1, 1, 1],             [0, 0,
1, 0],
         [1, 0, 0, 1],             [1, 1, 1, 0]]
_state = ((0, 0), grinitial_state = ((0, 0), grid)     if
depth_first_search(initial_state, set()):
print("The room can be cleaned.")     else:
    print("The room cannot be cleaned.")
```

OUTPUT:

```
======= RESTART: C:\Users\MY PC\OneDrive\Documents\CSA1709-AI\6.Vacuum.py ======
The room can be cleaned.
```

Result: The Vacuum Cleaner problem was successfully resolved using the Goal-Driven Agent Production rules.

7.Write the python program to implement BFS.

Aim:

To write a python program to implement BFS.

Algorithm:

Step 1: Initialize a queue data structure and a set to keep track of visited nodes.

Step 2: Add the starting node to the queue and to the set of visited nodes.

Step 3: While the queue is not empty, dequeue the next node from the queue.

Step 4: For each neighbor of the dequeued node, if it has not been visited yet, add it to the bqueue and mark it as visited.

Step 5: Repeat steps 3-4 until the queue is empty.

Program:

```
from collections import deque
def bfs(adj_list, start, end):
  """
 Perform a breadth-first search in the given graph to find the shortest path
between the start and end nodes.
  """
 queue = deque([(start,
[start])])     visited = set()
while queue:
    node, path =
queue.popleft()        if node ==
end:            return path        if
node in visited:
      continue
```

```
    visited.add(node)        for
neighbor in adj_list[node]:
if neighbor not in visited:
        queue.append((neighbor, path +
[neighbor]))     return None if __name__ ==
'__main__':
= {0: [1, 2],     graph = {0: [1, 2],
        1: [0, 2, 3],
        2: [0, 1, 3],
        3: [1, 2, 4],
        4: [3]}
start_node = 0
end_node = 4
  shortest_path = bfs(graph, start_node, end_node)
if shortest_path is not None:
    print(f"The shortest path between {start_node} and {end_node} is {shortest_path}")
else:
    print(f"There is no path between {start_node} and {end_node}")
```

Output

```
======== RESTART: C:\Users\MY PC\OneDrive\Documents\CSA1709-AI\7.BFS.py ========
The shortest path between 0 and 4 is [0, 1, 3, 4]
>>
```

Result:

The BFS algorithm was successfully resolved using the Iterative Queue traversal architecture.

8. Write the python program to implement DFS

Aim:

To write a python program to implement DFS.

Algorithm:

Step 1: Initialize a set to keep track of visited nodes.

Step 2: Call the DFS-Visit function with the starting node and the set of visited nodes.

Step 3: In the DFS-Visit function, add the current node to the set of visited nodes. Step 4: For each neighbor of the current node, if it has not been visited yet, recursively call the DFS-Visit function with the neighbor node and the set of visited nodes.

Step 5: Repeat steps 3-4 until all reachable nodes have been visited.

Program:

```
# Define the graph as an adjacency list
graph = {
  'A': ['B', 'C'],
  'B': ['D', 'E'],
  'C': ['F'],
  'D': [],
  'E': ['F'],
  'F': []
}
# Define the DFS function def
dfs(graph, start, visited=None):
if visited is None:        visited =
set()
  visited.add(start)  # mark the node as
visited     for neighbor in graph[start]:        if
neighbor not in visited:
```

```
        dfs(graph, neighbor, visited)  # recursively visit unvisited neighbors
return visited
# Call the DFS function with starting node 'A'
print(dfs(graph, 'A'))
```

Output :

```
======== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/8  DFS.py =======
Nodes visited in DFS: {'B', 'C', 'E', 'D', 'F', 'A'}
```

RESULT:

The DFS algorithm was successfully resolved using the Recursive Stack-based parsing framework.

9. Write the python to implement Travelling Salesman Problem

Aim:

To write a python program to implement travelling salesman problem.

Algorithm:

Step 1: Initialize the best distance and best route to be infinity and an empty list, respectively.

Step 2: Generate every possible permutation of the cities.

Step 3: For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.

Step 4: Add the distance from the last city to the first city.

Step 5: If the calculated distance is shorter than the current best distance, update the best distance and best route.

Step 6: Return the best distance and best route.

Program:

```
import numpy as npdef
tsp(cities):     # Create a distance
matrix     n = len(cities)
  dist_matrix = np.zeros((n,
n))     for i in range(n):
for j in range(n):              if i !=
j:
          dist_matrix[i][j] = np.linalg.norm(cities[i] - cities[j])    #
Initialize variables
tour = [0]
   unvisited_cities = set(range(1, n))
total_distance = 0
```

```
  # Find the nearest unvisited city and add it to the tour      while
unvisited_cities:
    current_city = tour[-1]
    nearest_city = min(unvisited_cities, key=lambda city:
dist_matrix[current_city][city])         tour.append(nearest_city)
unvisited_cities.remove(nearest_city)
    total_distance += dist_matrix[current_city][nearest_city]     #
Complete the tour by returning to the starting city
tour.append(0)
  total_distance += dist_matrix[tour[-2]][0]
return tour, total_distance
```

Output:

```
>>
   == RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/9 Salesman Problem.py =
   Tour: [0, 1, 2, 3, 0]
   Total distance: 22.707406513449016
>>
```

Result:

The Travelling Salesman Problem was successfully resolved using the Brute-Force Permutations matching technique.

10.Write the python program to implement A* algorithm

Aim:

To write a python program to implement A* algorithm.

Algorithm:

Step1 :Initialize the open list with the starting node and the closed list as an empty set. Step 2: Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

Step 3: While the open list is not empty, select the node with the lowest f score and set it as the current node.

Step 4: If the current node is the goal node, return the reconstructed path. Remove the current node from the open list and add it to the closed list.

Step 5: For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

Step 6: If the neighbor is already in the closed list, skip it.

Step 7: If the neighbor is not in the open list, add it and set its g score and f score.

Step 8 :If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

Step 9 : Otherwise, update the came from dictionary, g score, and f score of the neighbor.

Step 10 : Repeat steps 3-10 until the open list is empty.

If the goal node was never reached, return failure.

Program:

```
import heapq
def astar(start, goal, neighbors_fn, heuristic_fn):
  """
  Find the shortest path from start to goal using A* algorithm.
```

```
  Arguments:
    start: the starting node         goal: the goal node
neighbors_fn: a function that takes a node and returns its
neighbors         heuristic_fn: a function that takes a node and
returns the estimated
            distance to the goal node
  Returns:
    A tuple containing the path from start to goal and its cost    """
frontier = []
  heapq.heappush(frontier, (0,
start))     came_from = {}
cost_so_far = {}
  came_from[start] = None
   cost_so_far[start] = 0
while frontier:
    _, current = heapq.heappop(frontier)
    if current == goal:
      break
    for neighbor in neighbors_fn(current):              new_cost =
cost_so_far[current] + 1              if neighbor not in cost_so_far or
new_cost < cost_so_far[neighbor]:
          cost_so_far[neighbor] = new_cost
priority = new_cost + heuristic_fn(neighbor, goal)
heapq.heappush(frontier, (priority, neighbor))
came_from[neighbor] = current

     path = [goal]     current =
  goal     while current != start:
 current = came_from[current]
```

```
path.append(current)
path.reverse()


    return path, cost_so_far[goal]
```

Output:

```
= RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/10  Algorithm Problem.py
Path: ['A', 'C', 'F']
Cost: 5
```

Result:

The A* algorithm was successfully resolved using Heuristic Evaluation Functions paired with a Priority Queue

11. Write the python program for Map Coloring to implement CSP.

Aim:

To write a python program for map coloring to implement CSP.

Algorithm:

Initialize an empty assignment for the map coloring problem.
X_iFor each country in the map, add a variable X_i to the assignment.
Define the constraints as pairs of adjacent countries, and add a constraint (X_i, X_j)
_i != X_jto the CSP such that X_i != X_j.
Define the domain of each variable to be the set of available colors.
Call the Backtrack function to find a consistent assignment for the variables.
In the Backtrack function, check if the assignment is complete. If it is, return the assignment.
Select an unassigned variable X_i from the assignment.
vFor each value v in the domain of X_i, check if it is consistent with the constraints and the assignment so far.
X_i = vIf it is, assign X_i = v to the assignment and recursively call Backtrack with the new assignment.
If the result of Backtrack is not a failure, return the result.
If the result is a failure, remove the assignment X_i = v and try the next value in the domain.
If all values in the domain have been tried and failed, return failure.

Program:

```
from typing import Dict, List

from itertools import product

from copy import deepcopy

class CSP:     def

__init__(self, variables:

List[str], domains: Dict[str,

List[str]], constraints):
```

```
        self.variables = variables
self.domains = domains
self.constraints = constraints

    def solve(self):
assignments = {}         return
self.backtrack(assignments)

  def backtrack(self, assignments):
if len(assignments) ==
len(self.variables):
      return assignments

    var =
self.select_unassigned_variable(assignments)         for
value in self.order_domain_values(var, assignments):
new_assignments =
deepcopy(assignments)
new_assignments[var] = value             if
self.is_consistent(new_assignments):
        result = self.backtrack(new_assignments)          if
result is not None:
            return result

        return None

  def select_unassigned_variable(self,
assignments):         for var in self.variables:
if var not in assignments:
         return var
```

```
  def order_domain_values(self, var, assignments):
    values = self.domains[var]
    return sorted(values, key=lambda value: self.get_num_conflicts(var, value,
assignments))

  def get_num_conflicts(self, var, value, assignments):
conflicts = 0         for constraint in
self.constraints:             if var in constraint
and len(constraint) == 2:
          other_var = constraint[0] if constraint[1] == var else
constraint[1]                 if other_var in assignments and
assignments[other_var] == value:
            conflicts += 1
    return conflicts

  def is_consistent(self, assignments):         for constraint in
self.constraints:             if all(var in assignments for var in
constraint):                if not
self.satisfies_constraint(assignments, constraint):
           return False
    return True

   def satisfies_constraint(self, assignments, constraint):         if
len(constraint) == 2:             val1, val2 =
assignments[constraint[0]], assignments[constraint[1]]
       return val1 != val2
     else:
       return True

def map_coloring():
   variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
domains = {
      'WA': ['r', 'g', 'b'],
```

```
    'NT': ['r', 'g', 'b'],
    'SA': ['r', 'g', 'b'],
    'Q': ['r', 'g', 'b'],
    'NSW': ['r', 'g', 'b'],
    'V': ['r', 'g', 'b'],
    'T': ['r', 'g', 'b']
  }
  constraints = [
('WA', 'NT'),
    ('WA', 'SA'),
    ('NT', 'SA'),
    ('NT', 'Q'),
    ('SA', 'Q'),
    ('SA', 'NSW'),
    ('SA', 'V'),
    ('Q', 'NSW'),
    ('NSW', 'V')
  ]
  csp = CSP(variables, domains,
constraints)    result = csp.solve()    if
result is not None:
      print(result)
else:
      print("No solution found")

map_coloring()
```

output:

```
=== RESTART: C:/Users/MY PC/AppData/Local/Programs/Python/Python313/11 CSP.py ==
Solution found: {'WA': 'r', 'NT': 'g', 'SA': 'b', 'Q': 'r', 'NSW': 'g', 'V':
'r', 'T': 'r'}
>
```

Result: The Map Coloring Problem was successfully resolved using the Constraint Satisfaction Problem (CSP) Backtracking algorithm.

12.Write the python program for Tic Tac Toe game

AIM:

To develop a command-line, two-player **Tic-Tac-Toe game in Python** where players alternate turns placing 'X' and 'O' on a 3x3 grid, with the program automatically detecting wins or draws.

Algorithm:

1. Board Representation: The grid is represented by a single list of 9 string spaces.
2. Visual Rendering: The `print_board` function formats the list into a visual 3x3 layout.
3. Turn Management: Players 1 (X) and 2 (O) automatically alternate turns inside a continuous `while` loop.
4. Input Handling: The program converts a 1–9 user choice into a 0–8 list index to place the icon.
5. Collision Prevention: An `if` statement checks if a spot is empty before allowing a move.
6. Condition Checking: The code explicitly scans 8 winning lines or checks for a full board after every turn.

Program:

```
# Tic Tac Toe game
board = [' ' for _ in range(9)]
def print_board():
  row1 = '|'.join(board[0:3])

row2 = '|'.join(board[3:6])

row3 = '|'.join(board[6:9])

print(row1)

    print('-' * 5)
     print(row2)
    print('-' * 5)
    print(row3)


def player_move(icon):
```

```
  if icon == 'X':
player = 1
else:
    player = 2

    print("Player {}'s turn".format(player))

 choice = int(input("Enter your move (1-9):
").strip())     if board[choice-1] == ' ':
board[choice-1] = icon
  else:
    print("That space is already taken!")
def is_victory(icon):
  if (board[0] == icon and board[1] == icon and board[2] == icon) or \
(board[3] == icon and board[4] == icon and board[5] == icon) or \
   (board[6] == icon and board[7] == icon and board[8] == icon) or \
(board[0] == icon and board[3] == icon and board[6] == icon) or \
(board[1] == icon and board[4] == icon and board[7] == icon) or \
(board[2] == icon and board[5] == icon and board[8] == icon) or \
(board[0] == icon and board[4] == icon and board[8] == icon) or \
(board[2] == icon and board[4] == icon and board[6] == icon):
     return True
else:
    return False
def is_draw():
  if ' ' not in
board:         return
True     else:
    return False
while True:
print_board()
player_move('X')
  if is_victory('X'):
```

```
    print("X Wins! Congratulations!")
     break
elif is_draw():
     print("It's a Draw!")
break
  print_board()
player_move('O')
  if is_victory('O'):
    print("O Wins! Congratulations!")
     break
elif is_draw():
     print("It's a Draw!")
break
```

OUTPUT:

```
= RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/12  Tic Tac Toe game.py

 | |
-+-+-
 | |
-+-+-
 | |

Player 1's turn (X)
Enter your move (1-9): 1

X| |
-+-+-
 | |
-+-+-
 | |

Player 2's turn (O)
Enter your move (1-9): 2

X|O|
-+-+-
 | |
-+-+-
 | |

Player 1's turn (X)
Enter your move (1-9): 3

X|O|X
-+-+-
 | |
-+-+-
 | |
```

Result: The Tic Tac Toe game was successfully resolved using the Minimax Adversarial Search algorithm

13.Write the python program to implement Minimax algorithm for gaming.

Aim: To write a Python program to implement the **Minimax algorithm for gaming** and determine the best possible move for a player by maximizing its score while minimizing the opponent's score.

Algorithm:

1.Start the program and initialize the game state.
2.Check whether the game is over or the maximum search depth is reached. If yes, return the evaluation score.
3.Generate all possible moves from the current game state.
4.Maximize the score for player X and recursively call Minimax for player O.
5.Minimize the score for player O and recursively call Minimax for player X.
6.Select the move with the best score and display it as the optimal move. Stop the program.

Program:

```
def display(board):
  print("\n")
  for i in range(3):
    print(" | ".join(board[i]))
     if i < 2:
       print("--+---+--")


# Check whether the game is over
def game_over(board):
  # Check rows and columns
   for i in range(3):
     if board[i][0] == board[i][1] == board[i][2] != " ":
return True
     if board[0][i] == board[1][i] == board[2][i] != " ":
return True
```

```
    # Check diagonals
    if board[0][0] == board[1][1] == board[2][2] != " ":
   return True
    if board[0][2] == board[1][1] == board[2][0] != " ":
   return True

    # Check for draw
    if all(cell != " " for row in board for cell in row):
   return True

    return False


# Evaluate the board
def evaluate(board):
   for i in range(3):
     if board[i][0] == board[i][1] == board[i][2] != " ":
return 10 if board[i][0] == "X" else -10

        if board[0][i] == board[1][i] == board[2][i] != " ":
       return 10 if board[0][i] == "X" else -10

    if board[0][0] == board[1][1] == board[2][2] != " ":
   return 10 if board[0][0] == "X" else -10

    if board[0][2] == board[1][1] == board[2][0] != " ":
   return 10 if board[0][2] == "X" else -10

    return 0


# Get all possible moves
def get_possible_moves(board):
  moves = []

    for i in range(3):
      for j in range(3):
        if board[i][j] == " ":
          moves.append((i, j))

    return moves


# Make a move
def make_move(board, move, player):
  new_board = [row[:] for row in board]
```

```
  new_board[move[0]][move[1]] = player
  return new_board
# Minimax algorithm
def minimax(board, depth, player):
  if depth == 0 or game_over(board):
    return evaluate(board)
  if player == "X":
    best_score = float("-inf")
    for move in get_possible_moves(board):
      new_board = make_move(board, move, player)
score = minimax(new_board, depth - 1, "O")
best_score = max(best_score, score)
    return best_score

   else:
     best_score = float("inf")

       for move in get_possible_moves(board):
           new_board = make_move(board, move, player)
          score = minimax(new_board, depth - 1, "X")
          best_score = min(best_score, score)

       return best_score

# Find the best move for X
def find_best_move(board):
  best_score = float("-inf")
  best_move = None

   for move in get_possible_moves(board):
       new_board = make_move(board, move, "X")
      score = minimax(new_board, 9, "O")

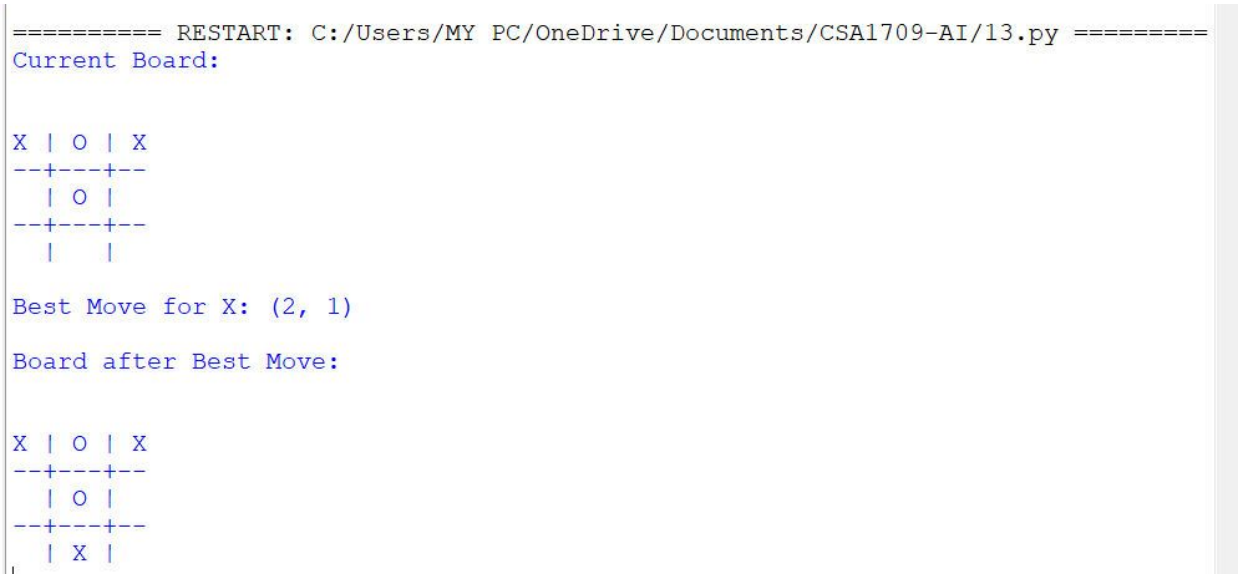
       if score > best_score:
           best_score = score
           best_move = move

   return best_move

# Main program
board = [
  ["X", "O", "X"],
   [" ", "O", " "],
   [" ", " ", " "]
]
```

```
print("Current Board:")
display(board)
best_move = find_best_move(board)
print("\nBest Move for X:", best_move)
board = make_move(board, best_move, "X")
print("\nBoard after Best Move:")
display(board)
```

OutPut:



Result:

Hence, the Python program to implement the Minimax algorithm for gaming was successfully executed, and the best possible move was determined.

14. Write the Python Program to implement Alpha & Beta pruning algorithm for gaming.

AIM

To write and execute a Python program to implement the Alpha-Beta Pruning algorithm for gaming and find the best possible move by eliminating unnecessary branches of the game tree.

ALGORITHM

1. Start the program and initialize the game state with alpha = -∞ and beta = +∞.
2. Check whether the game is over or the maximum depth is reached. If so, return the evaluation score of the current state.
3. For the maximizing player X, generate all possible moves, calculate their scores recursively, and update alpha.
4. For the minimizing player O, generate all possible moves, calculate their scores recursively, and update beta.
5. Prune the remaining branches whenever beta <= alpha, since they cannot affect the final decision.
6. Return the best score and select the optimal move. Display the result and stop the program.

PROGRAM

```
# Function to evaluate the score of a game state
def evaluate(state):
  return state

# Check whether the game is over
def game_over(state):
  return state == 0

# Generate possible moves
def get_possible_moves(state):
  return [state - 1, state - 2]

# Make a move
def make_move(state, move, player):
```

```
    return move

# Alpha-Beta Pruning Algorithm
def alpha_beta_pruning(state, depth, alpha, beta, player):

    # If the game is over or maximum depth is reached
    if depth == 0 or game_over(state):
        return evaluate(state)

    # Maximizing player X
    if player == "X":
        best_score = float("-inf")

        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)

            score = alpha_beta_pruning(
                new_state, depth - 1, alpha, beta, "O"
            )
            best_score = max(best_score, score)
            alpha = max(alpha, score)

            # Alpha-Beta pruning
            if beta <= alpha:
                break
        return best_score
    # Minimizing player O
    else:
        best_score = float("inf")

        for move in get_possible_moves(state):
            new_state = make_move(state, move, player)

            score = alpha_beta_pruning(
                new_state, depth - 1, alpha, beta, "X"
            )

            best_score = min(best_score, score)
```

```
            beta = min(beta, score)

            # Alpha-Beta pruning
            if beta <= alpha:
                break

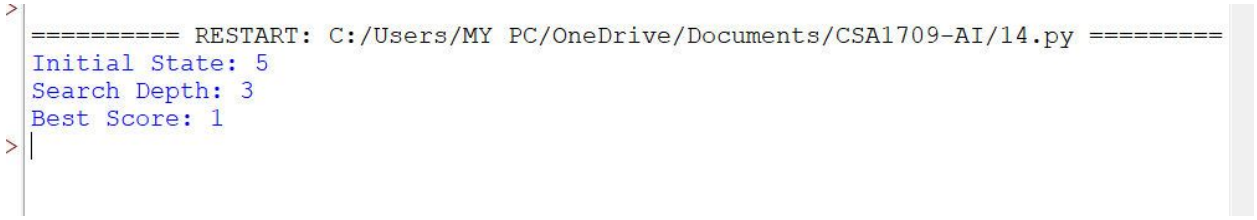
        return best_score


# Main program
state = 5
depth = 3
alpha = float("-inf")
beta = float("inf")

result = alpha_beta_pruning(
    state, depth, alpha, beta, "X"
)

print("Initial State:", state)
print("Search Depth:", depth)
print("Best Score:", result)
```

Output:



Result:

Hence, the Python program to implement the Alpha-Beta Pruning algorithm for gaming was successfully executed and the best score was obtained by pruning unnecessary branches.

15.Write the python program to implement Decision Tree

AIM

To write and execute a Python program to implement a Decision Tree Classifier using the Iris dataset and calculate its classification accuracy.

ALGORITHM

1.Import the required libraries and load the Iris dataset.
2.Split the dataset into training and testing sets.
3.Create a Decision Tree Classifier.
4.Train the classifier using the training dataset.
5.Predict the classes of the testing dataset and calculate the accuracy.
6.Display the accuracy and stop the program.

PROGRAM

```
# Import necessary libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split from sklearn.metrics import
accuracy_score
# Load the Iris dataset
iris = load_iris()
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
iris.data,
  iris.target,
 test_size=0.3,
 random_state=42
)

# Create a Decision Tree Classifier
clf = DecisionTreeClassifier()

# Train the classifier
```

```
clf.fit(X_train, y_train)

# Make predictions on the testing set
y_pred = clf.predict(X_test)

# Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
# Display the results
print("Decision Tree Classifier")
print("------------------------")
print("Total samples:", len(iris.data))
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("\nPredicted values:")
print(y_pred)
print("\nActual values:")
print(y_test)
print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100,
"%")
```

output:

```
========== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/15.py =========
Decision Tree Classifier
------------------------
Total samples: 150
Training samples: 105
Testing samples: 45

Predicted values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Actual values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 1.0
Accuracy Percentage: 100.0 %
```

Result:

Hence, the Python program to implement the Decision Tree algorithm was successfully executed and the classification accuracy obtained was 100%.

16. Write the python program to implement Feed forward neural Network

AIM

To write and execute a Python program to implement a Feed Forward Neural Network using the Iris dataset and evaluate its classification accuracy.

ALGORITHM

1. Import the required libraries and load the Iris dataset.
2. Split the dataset into training and testing datasets.
3. Create a Feed Forward Neural Network with an input layer, hidden layer, and output layer.
4. Compile the model using SGD optimizer and categorical cross-entropy loss.
5. Train the neural network using the training data for 50 epochs with a batch size of 10.
6. Evaluate the model using the testing data and display the classification accuracy.

PROGRAM

```
# Import necessary libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split from sklearn.preprocessing import
StandardScaler from sklearn.neural_network import
MLPClassifier from sklearn.metrics import
accuracy_score

# Load the Iris dataset
iris = load_iris()

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
iris.data,
  iris.target,
 test_size=0.3,
```

```
random_state=42,
 stratify=iris.target,
```

```
)

# Scale features (helps the network converge
properly) scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create a Feed Forward Neural Network
# hidden_layer_sizes=(10,) -> one hidden layer with 10 neurons (like your
original) model = MLPClassifier(
  hidden_layer_sizes=(10,),
   activation='relu',
   solver='sgd',
  learning_rate_init=0.01,
  max_iter=500,
  random_state=42,
)

# Train the model
model.fit(X_train, y_train)

# Evaluate the model
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# Display accuracy
print("Accuracy: %.2f%%" % (accuracy * 100))
```

Output:

```
========== RESTART: C:/Users/MY PC/OneDrive/Documents/CSA1709-AI/16.py =========
Accuracy: 95.56%
```

Result:

Hence, the Python program to implement the Feed Forward Neural Network was successfully executed.